Candidate Number: 48372

This Notebook is for providing answers to Question 2 (Neo4j Part) Only

Question (A)/(2A)

I have used the **arrows.app** to create a schematic diagram to visualise my graph database. Please check the file named "48372_Q(2A)_Arrows.app_schematic_graph" in the repository. However, the actual database creation is done in neo4j with the following code:

A.1 Connect to neo4j with my credentials.

```
from neo4j import GraphDatabase

uri = "neo4j+s://7c16e671.databases.neo4j.io"
username = "neo4j"
password = "Jm7W7Cxq1FcCtMq9OCUNRviXs4gmOFtrm5ky1WBQ1Zs"
```

A.2 Refring to the 1st assignment, create Entities.

Note: when creating Entities, there is no need to create properties at this stage, because I will modify properties when populating the data. But I do wish to specify Unique Identifiers for each table now.

```
class SetupConstraints:
    def __init__(self, uri, user, password):
        self.driver = GraphDatabase.driver(uri, auth=(user, password))
    def close(self):
        self.driver.close()
    def create_constraints(self):
        with self.driver.session() as session:
            session.run("CREATE CONSTRAINT IF NOT EXISTS FOR (c:Customer) REQUIRE c.email IS UNIQUE")
            session.run("CREATE CONSTRAINT IF NOT EXISTS FOR (o:Order) REQUIRE o.order_number IS UNIQUE")
            session.run("CREATE CONSTRAINT IF NOT EXISTS FOR (pm:PaymentMethod) REQUIRE pm.payment_method_id IS UNIQUE")
            session.run("CREATE CONSTRAINT IF NOT EXISTS FOR (cc:CreditCard) REQUIRE cc.credit_card_id IS UNIQUE")
            session.run("CREATE CONSTRAINT IF NOT EXISTS FOR (gc:GiftCard) REQUIRE gc.gift_card_id IS UNIQUE")
            session.run("CREATE CONSTRAINT IF NOT EXISTS FOR (cat:Category) REQUIRE cat.category_id IS UNIQUE")
            session.run("CREATE CONSTRAINT IF NOT EXISTS FOR (w:Wishlist) REQUIRE w.wishlist_id IS UNIQUE")
            session.run("CREATE CONSTRAINT IF NOT EXISTS FOR (d:Delivery) REQUIRE d.track_number IS UNIQUE")
            session.run("CREATE CONSTRAINT IF NOT EXISTS FOR (r:Return) REQUIRE r.ticket_number IS UNIQUE")
            session.run("CREATE CONSTRAINT IF NOT EXISTS FOR (p:Product) REQUIRE p.product_number IS UNIQUE")
            session.run("CREATE CONSTRAINT IF NOT EXISTS FOR (rev:Review) REQUIRE rev.review_number IS UNIQUE")
            session.run("CREATE CONSTRAINT IF NOT EXISTS FOR (oi:OrderedItem) REQUIRE oi.ordered_item_id IS UNIQUE")

setup = SetupConstraints(uri, username, password)
setup.create_constraints()
setup.close()
```

Question (B)/(2B)

In this section I will populate the data into the neo4j graph database. All data is hardcoded. The procedure to populate each table is the same. Basicly it could be divided into 2 parts: **inserting data and creating relationships**.

Both data insertion and relationship-building are done by this function **populate_xxx**. Inside this function I will include a list of data that I moved from my 1st assignment, which also includes corresponding foreign keys if there is any. This is because I will need to use them in building relationships between tables.

As for building relationships, I use **session.run**, which starts a cypher query. Inside the query, I use **MERGE** to create new nodes, this was done by searching **unique identifiers**. Because they are unique, they should never duplicate (but if there is duplication then the MERGE clause will detect and stop creating new ones). And when nodes are created, I can use **ON CREATE SET** to insert the rest of properties into nodes.

B.1 Populate Entity Customer

```
class PopulateCustomers:
    def __init__(self, uri, user, password):
        self.driver = GraphDatabase.driver(uri, auth=(user, password))
    def close(self):
        self.driver.close()
    def populate_customers(self):
        customers = [
            ('jane.smith@outlook.com', 'sdc789a2sc', 'Jane Smith', '1990-08-25', '0987654321', 'Canada', 'Toronto', 'Queen St', 'Sui
            ('alice.jones@gmail.com', 'sd89a5c33sc', 'Alice Jones', '1988-03-12', '1122334455', 'UK', 'London', 'Baker St', 'Flat 3
            ('bob.brown@outlook.com', 'asd787asdc2', 'Bob Brown', '1975-07-30', '6677889900', 'Australia', 'Sydney', 'George St', 'L
            ('carol.white@outlook.com', 'adeeewc455a', 'Carol White', '1995-11-20', '5566778899', 'Germany', 'Berlin', 'Alexanderplatz
            ('laura.green@gmail.com', 'gfhuyb34fgh', 'Laura Green', '1989-01-05', '2233445566', 'USA', 'Los Angeles', 'Sunset Blvd',
            ('chris.lee@outlook.com', 'w3egbbte4', 'Chris Lee', '1984-10-15', '9988776655', 'Canada', 'Vancouver', 'Main St', 'Suite
            ('emily.davis@gmail.com', 'h56frtfdc', 'Emily Davis', '1992-12-08', '3344556677', 'UK', 'Manchester', 'King St', 'Flat
            ('michael.jordan@yahoo.com', 'tvg45ty89', 'Michael Jordan', '1978-04-23', '4455667788', 'Australia', 'Melbourne', 'Collins
            ('susan.martin@hotmail.com', 'r4gtre2rf', 'Susan Martin', '1993-06-10', '6677889901', 'Germany', 'Hamburg', 'Reeperbahn',
            ('robert.jones@aol.com', 'a5trfgre12', 'Robert Jones', '1982-02-28', '2233445567', 'USA', 'Chicago', 'Michigan Ave', 'Ap
            ('nancy.parker@gmail.com', 'ae4htrgre', 'Nancy Parker', '1996-09-16', '1231231234', 'Canada', 'Montreal', 'Saint Laurent
            ('kevin.baker@gmail.com', '3gter5regr', 'Kevin Baker', '1987-12-04', '9988771122', 'UK', 'Liverpool', 'Bold St', 'Flat
            ('patricia.scott@yahoo.com', 'yh3rrf34y', 'Patricia Scott', '1979-03-29', '7766554433', 'Australia', 'Brisbane', 'Queen S
            ('thomas.anderson@outlook.com', 'qr5ttv44e', 'Thomas Anderson', '1983-11-12', '5566778822', 'Germany', 'Munich', 'Maximili
            ('diana.king@hotmail.com', 'yr43tg34', 'Diana King', '1994-05-07', '3344553322', 'France', 'Paris', 'Champs-Élysées', 'Fl
            ('mark.adams@gmail.com', 'tv45gre54t', 'Mark Adams', '1985-06-19', '6677884455', 'Italy', 'Rome', 'Via Veneto', 'Apt 4'
            ('gloria.williams@aol.com', 'at34rger', 'Gloria Williams', '1990-03-15', '3322115566', 'Spain', 'Madrid', 'Gran Via', 'S
            ('ryan.carter@gmail.com', 'gr43gt5gr', 'Ryan Carter', '1976-08-03', '7766552211', 'Portugal', 'Lisbon', 'Rua Augusta', '
            ('kimberly.allen@gmail.com', 'er43tr34t', 'Kimberly Allen', '1995-10-21', '3344557788', 'Netherlands', 'Amsterdam', 'Damra
            ('sara.watson@outlook.com', 'qt44rgre', 'Sara Watson', '1992-07-30', '6655443322', 'Sweden', 'Stockholm', 'Gamla Stan',
            ('frank.harris@gmail.com', 'gre34rt43', 'Frank Harris', '1980-04-14', '2233445568', 'Denmark', 'Copenhagen', 'Nørregade',
            ('matthew.reed@hotmail.com', '4trrg43t', 'Matthew Reed', '1988-09-25', '7766554499', 'Norway', 'Oslo', 'Karl Johans Gate
            ('julia.moore@gmail.com', 'qtr5t4grr', 'Julia Moore', '1991-01-11', '3344552233', 'Switzerland', 'Zurich', 'Bahnhofstrasse
            ('philip.evans@yahoo.com', 'wq23r4tf', 'Philip Evans', '1986-02-20', '4455668822', 'Belgium', 'Brussels', 'Avenue Louise'
            ('oliver.taylor@gmail.com', 'qwef34gtr', 'Oliver Taylor', '1993-06-15', '9988772211', 'Finland', 'Helsinki', 'Esplanadi',
            ('hannah.collins@aol.com', 'wqt43rgt', 'Hannah Collins', '1989-12-20', '5544336677', 'Ireland', 'Dublin', 'OConnell Stree
            ('timothy.hall@gmail.com', 'rtg45tgr3', 'Timothy Hall', '1975-03-03', '4433221100', 'Austria', 'Vienna', 'Kärntner Strass
            ('anne.wilson@hotmail.com', 'tr34yt43', 'Anne Wilson', '1987-07-04', '2233445599', 'New Zealand', 'Wellington', 'Lambton
            ('elizabeth.james@gmail.com', '3tg4rger', 'Elizabeth James', '1991-10-30', '1231239988', 'Singapore', 'Singapore', 'Orchar
            ('steven.wright@outlook.com', 'h45grrt34', 'Steven Wright', '1984-09-17', '9988775533', 'South Korea', 'Seoul', 'Gangnam
            ('maria.hill@gmail.com', 'tgr34tre4', 'Maria Hill', '1982-05-08', '3344551199', 'Japan', 'Tokyo', 'Shibuya', 'Suite 15',
            ('henry.young@gmail.com', 'r4tger45g', 'Henry Young', '1992-11-13', '6677889902', 'China', 'Beijing', 'Wangfujing St', '
            ('daniel.clark@yahoo.com', '4tgger54r', 'Daniel Clark', '1979-04-21', '1122334456', 'Russia', 'Moscow', 'Tverskaya St',
            ('isabella.hernandez@gmail.com', 'gtrf4grt4', 'Isabella Hernandez', '1995-02-06', '6655449988', 'Brazil', 'Rio de Janeiro
            ('victoria.mitchell@gmail.com', 'grt4gr35', 'Victoria Mitchell', '1986-12-09', '4433221101', 'Argentina', 'Buenos Aires',
            ('aaron.morris@yahoo.com', 'qtr5gtr', 'Aaron Morris', '1989-07-22', '2233445577', 'South Africa', 'Cape Town', 'Long St
            ('joseph.cook@gmail.com', 'tgr43ytr', 'Joseph Cook', '1983-10-03', '6677881133', 'Israel', 'Tel Aviv', 'Dizengoff St',
            ('megan.foster@outlook.com', 'tre43tgr', 'Megan Foster', '1994-05-18', '9988773355', 'Mexico', 'Mexico City', 'Reforma',
            ('michael.brown@gmail.com', 'gtrt4fgt', 'Michael Brown', '1985-02-15', '6677889903', 'USA', 'San Francisco', 'Market St'
        ]

        with self.driver.session() as session:
            for c in customers:
                session.run(
                    """
                    MERGE (cust:Customer {email:$email})
                    ON CREATE SET cust.password=$password, cust.name=$name, cust.birthdate=$birthdate,
                                  cust.phonenumber=$phonenumber, cust.country=$country, cust.city=$city,
                                  cust.street=$street, cust.building_info=$building_info, cust.postcode=$postcode
                    """,
                    email=c[0], password=c[1], name=c[2], birthdate=c[3], phonenumber=c[4],
                    country=c[5], city=c[6], street=c[7], building_info=c[8], postcode=c[9]
                )

cust_pop = PopulateCustomers(uri, username, password)
cust_pop.populate_customers()
cust_pop.close()
```

B.2 Populate Entity Order

```
class PopulateOrders:
    def __init__(self, uri, user, password):
        self.driver = GraphDatabase.driver(uri, auth=(user, password))
    def close(self):
        self.driver.close()
    def populate_orders(self):
        orders = [
```

```
                (1, '2024-01-15', '5%', '699.99,799.99', 1499.98, 1421.98, 'john.doe@gmail.com'),
                (2, '2024-02-20', '10%', '1299.99,99.99', 1399.98, 1259.98, 'jane.smith@outlook.com'),
                (3, '2024-03-10', '0%', '199.99,299.99', 499.98, 499.98, 'alice.jones@gmail.com'),
                (4, '2024-04-25', '15%', '79.99,399.99', 479.98, 407.98, 'bob.brown@outlook.com'),
                (5, '2024-05-30', '0%', '178.99,49.99', 228.98, 228.98, 'carol.white@outlook.com'),
                (6, '2024-06-15', '20%', '1299.99,59.97', 1359.96, 1087.97, 'laura.green@gmail.com'),
                (7, '2024-07-22', '5%', '1996.96,39.99,49.99', 2086.94, 1982.593, 'chris.lee@outlook.com'),
                (8, '2024-08-18', '10%', '199.99,99.99', 299.98, 269.98, 'emily.davis@gmail.com'),
                (9, '2024-09-05', '0%', '19.99,89.99,19.99', 129.97, 129.97, 'michael.jordan@yahoo.com'),
                (10, '2024-10-10', '25%', '39.99,149.99', 189.99, 142.49, 'susan.martin@hotmail.com'),
                (11, '2024-01-22', '12%', '299.99,799.99,79.78', 1179.76, 1037.98, 'nancy.parker@gmail.com'),
                (12, '2024-02-15', '8%', '29.99,69.99', 99.98, 91.98, 'kevin.baker@gmail.com'),
                (13, '2024-03-30', '3%', '39.99,39.99', 79.98, 77.58, 'patricia.scott@yahoo.com'),
                (14, '2024-04-05', '0%', '14.99,49.99', 64.98, 64.98, 'frank.harris@gmail.com'),
                (15, '2024-05-11', '20%', '99.99,199.99', 299.98, 239.98, 'isabella.hernandez@gmail.com'),
                (16, '2024-06-18', '10%', '49.98,89.99', 139.97, 125.97, 'ryan.carter@gmail.com'),
                (17, '2024-07-21', '0%', '34.99,299.99', 334.98, 334.98, 'kimberly.allen@gmail.com'),
                (18, '2024-08-12', '5%', '49.99,129.99', 179.98, 170.98, 'hannah.collins@aol.com'),
                (19, '2024-09-07', '7%', '59.99,499.99', 559.98, 520.78, 'thomas.anderson@outlook.com'),
                (20, '2024-10-02', '10%', '159.96,79.99,29.99', 269.94, 242.95, 'elizabeth.james@gmail.com'),
                (21, '2024-01-19', '0%', '199.99,49.99', 249.98, 249.98, 'matthew.reed@hotmail.com'),
                (22, '2024-02-14', '15%', '799.99,89.99', 889.98, 756.48, 'mark.adams@gmail.com'),
                (23, '2024-03-16', '5%', '299.99,14.99', 314.98, 299.23, 'julia.moore@gmail.com'),
                (24, '2024-04-20', '12%', '79.99,9.99', 89.98, 79.18, 'sara.watson@outlook.com'),
                (25, '2024-05-25', '0%', '39.99,39.98,49.99', 129.96, 129.96, 'timothy.hall@gmail.com'),
                (26, '2024-06-05', '8%', '29.99,199.99', 229.98, 211.58, 'julia.moore@gmail.com'),
                (27, '2024-07-18', '20%', '29.99,299.99', 329.98, 263.98, 'joseph.cook@gmail.com'),
                (28, '2024-08-22', '3%', '89.99,69.99', 159.98, 155.18, 'aaron.morris@yahoo.com'),
                (29, '2024-09-11', '5%', '49.99,89.99,19.99', 159.97, 151.97, 'victoria.mitchell@gmail.com'),
                (30, '2024-10-30', '10%', '399.99,299.99', 699.98, 629.98, 'oliver.taylor@gmail.com'),
                (31, '2024-01-12', '7%', '129.99,39.99', 169.98, 158.08, 'joseph.cook@gmail.com'),
                (32, '2024-02-28', '10%', '69.99,29.99', 99.98, 89.98, 'michael.brown@gmail.com'),
                (33, '2024-03-01', '20%', '99.99,49.99', 149.98, 119.98, 'maria.hill@gmail.com'),
                (34, '2024-03-28', '0%', '199.99,29.99', 229.98, 229.98, 'diana.king@hotmail.com'),
                (35, '2024-04-10', '8%', '249.99,49.99', 299.98, 275.98, 'steven.wright@outlook.com'),
                (36, '2024-05-13', '5%', '39.99,19.99', 59.98, 56.98, 'henry.young@gmail.com'),
                (37, '2024-06-25', '0%', '59.97,24.99,498.98', 583.94, 583.94, 'megan.foster@outlook.com'),
                (38, '2024-07-20', '15%', '49.99,29.99', 79.98, 67.98, 'julia.moore@gmail.com'),
                (39, '2024-08-14', '5%', '29.99,9.99', 39.98, 37.98, 'timothy.hall@gmail.com'),
                (40, '2024-09-29', '20%', '19.99,99.99', 119.98, 95.98, 'julia.moore@gmail.com'),
                (41, '2024-10-15', '0%', '9.99,199.99,24.99', 234.97, 234.97, 'joseph.cook@gmail.com'),
                (42, '2024-01-01', '30%', '199.99,79.99', 279.98, 195.99, 'thomas.anderson@outlook.com'),
                (43, '2024-02-05', '10%', '129.99,49.99', 179.98, 161.98, 'laura.green@gmail.com'),
                (44, '2024-03-08', '5%', '49.99,129.99', 179.98, 170.98, 'kevin.baker@gmail.com'),
                (45, '2024-04-15', '0%', '50.00,59.99', 109.99, 109.99, 'sara.watson@outlook.com'),
                (46, '2024-05-20', '10%', '29.99,29.99', 59.98, 53.98, 'frank.harris@gmail.com')
        ]

        with self.driver.session() as session:
            for o in orders:
                session.run(
                    """
                    MERGE (ord:Order {order_number:$order_number})
                    ON CREATE SET ord.order_date=$order_date, ord.deduction=$deduction, ord.subtotal=$subtotal,
                                  ord.sum_subtotal=$sum_subtotal, ord.grand_total=$grand_total, ord.customer_email=$em
                    """,
                    order_number=o[0], order_date=o[1], deduction=o[2], subtotal=o[3],
                    sum_subtotal=o[4], grand_total=o[5], email=o[6]
                )

ord_pop = PopulateOrders(uri, username, password)
ord_pop.populate_orders()
ord_pop.close()
```

B.3 Populate Entity Product

```
class PopulateProducts:
    def __init__(self, uri, user, password):
        self.driver = GraphDatabase.driver(uri, auth=(user, password))
    def close(self):
        self.driver.close()
    def populate_products(self):
        products = [
            (50001, 'Smartphone', 'TechBrand', 'Black', 0.2, 699.99, '15x7x0.8 cm', 30, 'Latest smartphone with cutting-edge feat
            (50002, 'Laptop', 'CompTech', 'Silver', 1.5, 1299.99, '35x24x2 cm', 20, 'Powerful laptop for professionals', 'False',
```

(50003, 'Headphones', 'SoundCo', 'Black', 0.3, 199.99, '15x15x5 cm', 25, 'Wireless headphones with noise cancellation
(50004, 'Microwave Oven', 'HomeAppliance', 'Silver', 15.0, 399.99, '50x40x30 cm', 15, 'Compact microwave oven for qu
(50005, 'Blender', 'KitchenAid', 'Red', 3.0, 89.99, '40x20x20 cm', 45, 'High-speed blender for smoothies', 'True', '
(50006, 'Refrigerator', 'FridgePro', 'White', 60.0, 1299.99, '180x70x70 cm', 10, 'Large refrigerator with energy effi
(50007, 'Sofa', 'ComfortHome', 'Brown', 100.0, 499.99, '200x80x90 cm', 5, 'Stylish sofa for living room', 'True', '
(50008, 'Coffee Table', 'ModernLiving', 'Glass', 20.0, 199.99, '120x60x40 cm', 30, 'Elegant coffee table for home',
(50009, 'T-Shirt', 'FashionBrand', 'Blue', 0.2, 19.99, 'L', 100, 'Casual T-shirt for everyday wear', 'True', '1 yea
(50010, 'Jeans', 'DenimCo', 'Black', 0.5, 39.99, '32', 50, 'Comfortable jeans for all occasions', 'False', '1 year'
(50011, 'Watch', 'TimeBrand', 'Gold', 0.1, 299.99, '10x10 cm', 15, 'Elegant wristwatch with leather strap', 'True',
(50012, 'Sunscreen', 'SkinCare', 'White', 0.2, 29.99, '15x5 cm', 35, 'SPF 50 sunscreen for skin protection', 'True'
(50013, 'Yoga Mat', 'FitGear', 'Purple', 1.0, 39.99, '180x60 cm', 25, 'Non-slip yoga mat for workouts', 'True', '1
(50014, 'Fiction Novel', 'BookPublisher', 'N/A', 0.5, 14.99, '20x13 cm', 50, 'Bestselling fiction novel', 'True', '1
(50015, 'Fitness Tracker', 'FitTrack', 'Black', 0.1, 99.99, '5x5 cm', 30, 'Wearable fitness tracker for health moni
(50016, 'Soccer Ball', 'SportCo', 'White', 0.4, 24.99, '22 cm', 40, 'Official size soccer ball for matches', 'True
(50017, 'Board Game', 'FunGames', 'Multicolor', 1.5, 34.99, '30x30x5 cm', 20, 'Exciting board game for family fun',
(50018, 'Car Seat Cover', 'AutoComfort', 'Black', 1.0, 49.99, 'Universal', 30, 'Protective cover for car seats', '1
(50019, 'Garden Tools Set', 'GreenThumb', 'Green', 2.0, 59.99, '40x30 cm', 25, 'Complete set of gardening tools',
(50020, 'Camping Lantern', 'OutdoorGear', 'Yellow', 1.5, 39.99, '30 cm', 35, 'Bright lantern for camping trips', '1
(50021, 'Action Camera', 'ActionPro', 'Black', 0.4, 199.99, '10x5x5 cm', 18, 'Waterproof action camera for adventure
(50022, 'Washing Machine', 'CleanTech', 'White', 75.0, 799.99, '85x60x60 cm', 12, 'High-efficiency washing machine',
(50023, 'Vacuum Cleaner', 'CleanPro', 'Silver', 6.0, 299.99, '30x30x110 cm', 30, 'Powerful vacuum cleaner for home',
(50024, 'Coffee Maker', 'BrewMaster', 'Black', 5.0, 79.99, '30x20x20 cm', 25, 'Programmable coffee maker for home',
(50025, 'Electric Kettle', 'KitchenAid', 'Silver', 1.0, 39.99, '25x15 cm', 35, 'Fast boiling electric kettle', 'True
(50026, 'Frying Pan', 'Cookware', 'Black', 1.0, 29.99, '30x30 cm', 40, 'Non-stick frying pan for cooking', 'True',
(50027, 'Cookbook', 'Foodie', 'N/A', 0.5, 29.99, '25x20 cm', 35, 'Delicious recipes for home cooking', 'True', '1
(50028, 'Tennis Racket', 'SportPro', 'Red', 0.4, 89.99, '70x25x5 cm', 30, 'High-quality tennis racket', 'True', '2
(50029, 'Fishing Rod', 'FishGear', 'Green', 1.0, 49.99, '210 cm', 25, 'Durable fishing rod for anglers', 'False',
(50030, 'Bicycle', 'BikePro', 'Blue', 12.0, 399.99, '170 cm', 10, 'Mountain bike for all terrains', 'True', '2 yea
(50031, 'Tent', 'CampingCo', 'Green', 3.0, 129.99, '250x200x150 cm', 15, 'Waterproof tent for camping', 'True', '1
(50032, 'Sleeping Bag', 'OutdoorGear', 'Red', 1.0, 69.99, '210 cm', 20, 'Comfortable sleeping bag for camping', 'Fa
(50033, 'GPS Device', 'NavTech', 'Black', 0.5, 99.99, '10x10 cm', 15, 'Accurate GPS device for navigation', 'True',
(50034, 'Leather Jacket', 'FashionCo', 'Black', 1.2, 199.99, 'M', 25, 'Stylish leather jacket for men', 'True', '2
(50035, 'Smart Watch', 'TechBrand', 'Silver', 0.2, 249.99, '4x4 cm', 40, 'Smart watch with fitness tracking', 'True
(50036, 'Wireless Charger', 'ChargeCo', 'Black', 0.1, 39.99, '10x10 cm', 30, 'Fast wireless charger for smartphones'
(50037, 'Puzzle', 'FunGames', 'Multicolor', 0.8, 19.99, '30x30 cm', 20, 'Challenging jigsaw puzzle', 'False', '1 yea
(50038, 'Hair Dryer', 'BeautyPro', 'Pink', 0.5, 49.99, '25x10 cm', 35, 'Powerful hair dryer for styling', 'True',
(50039, 'Face Cream', 'BeautyBrand', 'White', 0.2, 29.99, '50 ml', 30, 'Moisturizing face cream', 'True', '1 year',
(50040, 'Vitamins', 'HealthPlus', 'N/A', 0.1, 19.99, '60 capsules', 35, 'Daily vitamins for health', 'True', '1 yea
(50041, 'Notebook', 'OfficeCo', 'Blue', 0.3, 9.99, 'A4', 45, 'Spiral-bound notebook', 'True', '1 year', None, '0012'
(50042, 'Printer', 'PrintCo', 'Black', 5.0, 199.99, '40x30x15 cm', 25, 'Color printer for home and office', 'True',
(50043, 'Office Chair', 'ComfortOffice', 'Gray', 10.0, 129.99, '100x60x90 cm', 15, 'Ergonomic office chair', 'True',
(50044, 'Router', 'NetTech', 'Black', 0.5, 49.99, '25x20x5 cm', 35, 'High-speed wireless router', 'True', '1 year',
(50045, 'Video Game', 'GameBrand', 'N/A', 0.08, 50.00, 'N/A', 30, 'Latest video game for console', 'True', '1 year
(50046, 'Wireless Mouse', 'TechCo', 'Black', 0.1, 29.99, '10x6 cm', 40, 'Ergonomic wireless mouse', 'True', '1 year
(50047, 'Camera', 'PhotoTech', 'Black', 1.0, 799.99, '15x10x5 cm', 10, 'DSLR camera with high resolution', 'True',
(50048, 'Smart Speaker', 'AudioBrand', 'White', 0.8, 99.99, '15x15x10 cm', 20, 'Smart speaker with voice assistant',
(50049, 'Desk', 'OfficeBrand', 'Brown', 15.0, 299.99, '140x70x75 cm', 15, 'Wooden desk for office', 'True', '2 year
(50050, 'Tablet', 'GadgetCo', 'Black', 0.5, 399.99, '24x17x0.5 cm', 25, 'Portable tablet for work and play', 'True'
(50051, 'Garden Chair', 'OutdoorBrand', 'Green', 5.0, 49.99, '60x60x90 cm', 35, 'Comfortable chair for outdoor use',
(50052, 'Smart Bulb', 'LightTech', 'White', 0.1, 19.99, '10x10 cm', 40, 'Smart bulb with app control', 'True', '1
(50053, 'Baking Set', 'KitchenCraft', 'Multicolor', 1.0, 39.99, '30x30 cm', 30, 'Complete baking set for enthusiasts
(50054, 'Electric Toothbrush', 'OralCare', 'Blue', 0.5, 99.99, '20x5 cm', 25, 'Electric toothbrush with timer', 'Tru
(50055, 'Home Security Camera', 'SecureHome', 'Black', 0.3, 89.99, '10x10 cm', 30, 'Smart security camera with nigh
(50056, 'Heater', 'HeatPro', 'White', 10.0, 149.99, '40x40x80 cm', 15, 'Portable heater for winter', 'True', '1 yea
(50057, 'Air Conditioner', 'CoolTech', 'White', 30.0, 799.99, '75x50x30 cm', 10, 'Energy-efficient air conditioner',
(50058, 'Bluetooth Speaker', 'SoundBrand', 'Black', 1.0, 69.99, '20x10 cm', 25, 'Portable Bluetooth speaker with dee
(50059, 'Electric Fan', 'AirFlow', 'White', 2.0, 39.99, '40x40x100 cm', 30, 'Cooling fan for home', 'True', '1 yea
(50060, 'Window Blinds', 'HomeDecor', 'Beige', 1.0, 49.99, '120x100 cm', 20, 'Adjustable window blinds', 'True', '1
(50061, 'Air Fryer', 'CookMaster', 'Black', 5.0, 199.99, '30x30x30 cm', 15, 'Healthy air fryer for cooking', 'True'
(50062, 'Pressure Cooker', 'KitchenAid', 'Silver', 4.0, 89.99, '25x25 cm', 25, 'Electric pressure cooker', 'True', '
(50063, 'Drone', 'FlyTech', 'White', 0.8, 299.99, '30x30 cm', 20, 'High-performance drone with camera', 'True', '2
(50064, 'E-Reader', 'ReadPro', 'Black', 0.3, 129.99, '15x10 cm', 30, 'Portable e-reader for books', 'True', '1 year
(50065, 'Video Projector', 'VisionPro', 'Black', 3.0, 499.99, '30x30 cm', 15, 'High-resolution video projector', 'Tru
(50066, 'Smart Home Hub', 'SmartHome', 'White', 0.5, 79.99, '10x10 cm', 40, 'Hub for connecting smart devices', 'T
(50067, 'Webcam', 'TechCam', 'Black', 0.3, 49.99, '10x10 cm', 30, 'HD webcam for video calls', 'True', '1 year',
(50068, 'Cordless Drill', 'ToolBrand', 'Red', 1.5, 89.99, '25x10 cm', 25, 'Powerful cordless drill for DIY', 'True'
(50069, 'Hammer', 'ToolPro', 'Gray', 1.0, 14.99, '30 cm', 40, 'Durable hammer for construction', 'True', '1 year',
(50070, 'Tape Measure', 'ToolKit', 'Yellow', 0.3, 9.99, '5 m', 35, 'Compact tape measure for measuring', 'True', '
(50071, 'Garden Hose', 'OutdoorBrand', 'Green', 2.0, 19.99, '30 m', 30, 'Durable garden hose for watering', 'True',
(50072, 'Patio Table', 'OutdoorCo', 'Brown', 15.0, 199.99, '120x80 cm', 15, 'Sturdy patio table for outdoor dining'
(50073, 'Outdoor Grill', 'BBQPro', 'Black', 10.0, 299.99, '80x60 cm', 10, 'Portable grill for barbecues', 'True', '
(50074, 'Camping Stove', 'OutdoorGear', 'Silver', 2.0, 69.99, '40x30 cm', 25, 'Compact camping stove', 'True', '1 y
(50075, 'Hiking Backpack', 'TravelPro', 'Blue', 1.5, 89.99, '50x30x15 cm', 35, 'Durable backpack for hiking', 'True'
(50076, 'Inflatable Kayak', 'WaterSports', 'Yellow', 8.0, 299.99, '300 cm', 15, 'Inflatable kayak for water sports',
(50077, 'Fishing Tackle Box', 'FishGear', 'Green', 2.0, 39.99, '50x30x20 cm', 25, 'Spacious tackle box for fishing
(50078, 'Thermos Flask', 'TravelBrand', 'Silver', 0.5, 29.99, '25x10 cm', 40, 'Insulated flask for hot drinks', 'Tr
(50079, 'Camping Hammock', 'OutdoorCo', 'Green', 1.0, 49.99, '250 cm', 20, 'Lightweight hammock for relaxation', 'Tr

```
                (50080, 'Portable Charger', 'ChargeTech', 'Black', 0.2, 29.99, '10x5 cm', 35, 'Compact charger for devices', 'True',
                (50081, 'Leather Wallet', 'FashionBrand', 'Brown', 0.1, 49.99, '10x10 cm', 30, 'Stylish leather wallet', 'True', '1
                (50082, 'Smartphone Case', 'TechBrand', 'Black', 0.1, 19.99, '15x7 cm', 25, 'Protective case for smartphones', 'True
                (50083, 'Action Figure', 'ToyBrand', 'Multicolor', 0.5, 24.99, '20x10 cm', 40, 'Collectible action figure', 'True',
                (50084, 'Puzzle Mat', 'FunGames', 'Blue', 1.0, 29.99, '150x100 cm', 35, 'Non-slip mat for puzzles', 'True', '1 yea
                (50085, 'Balloon Set', 'PartyBrand', 'Multicolor', 0.2, 9.99, 'Variety', 50, 'Set of colorful balloons', 'True', '1
                (50086, 'Kids Bike', 'ToyBrand', 'Pink', 5.0, 99.99, '100 cm', 20, 'Safe bike for kids', 'True', '2 years', 601,
                (50087, 'Stroller', 'BabyBrand', 'Gray', 10.0, 199.99, '100x60x100 cm', 15, 'Convenient stroller for babies', 'True',
                (50088, 'Baby Monitor', 'BabyTech', 'White', 0.5, 79.99, '10x10 cm', 25, 'Smart baby monitor with camera', 'True',
                (50089, 'Diaper Bag', 'BabyCare', 'Blue', 0.5, 49.99, '40x30x10 cm', 30, 'Stylish diaper bag for parents', 'True',
                (50090, 'Kids Tablet', 'GadgetCo', 'Pink', 0.5, 129.99, '24x17x0.5 cm', 20, 'Tablet designed for kids', 'True', '2
                (50091, 'Activity Gym', 'BabyFun', 'Multicolor', 2.0, 59.99, '100 cm', 15, 'Interactive gym for babies', 'True', '1
                (50092, 'Toddler Shoes', 'FashionKids', 'Red', 0.5, 29.99, 'Size 5', 40, 'Comfortable shoes for toddlers', 'True',
                (50093, 'Kids Art Set', 'CreativeKids', 'Multicolor', 0.5, 24.99, '40x30 cm', 30, 'Complete art set for kids', 'Tr
                (50094, 'Kids Bed', 'FurnitureCo', 'Blue', 25.0, 299.99, '130x70 cm', 10, 'Comfortable bed for kids', 'True', '2 y
                (50095, 'Bike Helmet', 'SafetyGear', 'Black', 0.5, 49.99, 'N/A', 25, 'Protective helmet for biking', 'True', '1 yea
                (50096, 'Kids Puzzle', 'FunGames', 'Multicolor', 0.5, 19.99, '30x30 cm', 35, 'Educational puzzle for kids', 'True',
                (50097, 'Baby Clothes', 'FashionBaby', 'Pink', 0.5, 29.99, 'Variety', 30, 'Comfortable clothes for babies', 'True',
                (50098, 'Playmat', 'BabyFun', 'Multicolor', 2.0, 49.99, '150x150 cm', 15, 'Soft playmat for babies', 'True', '1 yea
                (50099, 'Kids Swim Set', 'SwimBrand', 'Blue', 0.5, 39.99, 'Variety', 20, 'Complete swim set for kids', 'True', '2
                (50100, 'Kids Bike Accessories', 'BikeFun', 'Black', 0.3, 19.99, 'Variety', 30, 'Accessories for kids bikes', 'True'
        ]

        with self.driver.session() as session:
            for p in products:
                session.run(
                    """
                    MERGE (pr:Product {product_number:$product_number})
                    ON CREATE SET pr.short_name=$short_name, pr.brand=$brand, pr.color=$color, pr.weight=$weight,
                                  pr.price=$price, pr.dimensions=$dimensions, pr.stock_quantity=$stock_quantity,
                                  pr.textual_description=$textual_description, pr.is_available=$is_available,
                                  pr.warranty_period=$warranty_period, pr.ticket_number=$ticket_number,
                                  pr.category_id=$category_id, pr.wishlist_id=$wishlist_id
                    """,
                    product_number=p[0], short_name=p[1], brand=p[2], color=p[3], weight=p[4], price=p[5],
                    dimensions=p[6], stock_quantity=p[7], textual_description=p[8], is_available=p[9],
                    warranty_period=p[10], ticket_number=p[11], category_id=p[12], wishlist_id=p[13]
                )

prod_pop = PopulateProducts(uri, username, password)
prod_pop.populate_products()
prod_pop.close()
```

B.4 Populate Entity Category

```
class PopulateCategories:
    def __init__(self, uri, user, password):
        self.driver = GraphDatabase.driver(uri, auth=(user, password))
    def close(self):
        self.driver.close()
    def populate_categories(self):
        categories = [
            ('0001', 'Electronics'),
            ('0002', 'Home Appliances'),
            ('0003', 'Fashion'),
            ('0004', 'Health & Beauty'),
            ('0005', 'Books'),
            ('0006', 'Sports & Outdoors'),
            ('0007', 'Toys & Games'),
            ('0008', 'Automotive'),
            ('0009', 'Furniture'),
            ('0010', 'Groceries'),
            ('0011', 'Music & Entertainment'),
            ('0012', 'Office Supplies'),
            ('0013', 'Pet Supplies'),
            ('0014', 'Jewelry & Accessories'),
            ('0015', 'Garden & Outdoor')
        ]
        with self.driver.session() as session:
            for cat in categories:
                session.run(
                    """
                    MERGE (c:Category {category_id:$category_id})
                    ON CREATE SET c.category_type=$category_type
                    """,
```

```
                                category_id=cat[0],  category_type=cat[1]
                        )

cat_pop  =  PopulateCategories(uri,  username,  password)
cat_pop.populate_categories()
cat_pop.close()
```

B.5 Populate Entity Wishlist

```
class  PopulateWishlist:
        def  __init__(self,  uri,  user,  password):
                self.driver  =  GraphDatabase.driver(uri,  auth=(user,  password))
        def  close(self):
                self.driver.close()
        def  populate_wishlist(self):
                wishlists  =  [
                        (701,  3,  'julia.moore@gmail.com'),
                        (702,  1,  'timothy.hall@gmail.com'),
                        (703,  2,  'isabella.hernandez@gmail.com'),
                        (704,  5,  'julia.moore@gmail.com'),
                        (705,  2,  'carol.white@outlook.com'),
                        (706,  4,  'laura.green@gmail.com'),
                        (707,  1,  'julia.moore@gmail.com'),
                        (708,  3,  'emily.davis@gmail.com'),
                        (709,  2,  'michael.jordan@yahoo.com'),
                        (710,  6,  'susan.martin@hotmail.com'),
                        (711,  2,  'nancy.parker@gmail.com'),
                        (712,  4,  'kevin.baker@gmail.com'),
                        (713,  1,  'timothy.hall@gmail.com'),
                        (714,  5,  'frank.harris@gmail.com'),
                        (715,  3,  'isabella.hernandez@gmail.com'),
                        (716,  2,  'ryan.carter@gmail.com'),
                        (717,  1,  'kimberly.allen@gmail.com'),
                        (718,  6,  'hannah.collins@aol.com'),
                        (719,  3,  'elizabeth.james@gmail.com'),
                        (720,  4,  'thomas.anderson@outlook.com')
                ]
                with  self.driver.session()  as  session:
                        for  w  in  wishlists:
                                session.run(
                                        """
                                        MERGE  (wl:Wishlist  {wishlist_id:$wishlist_id})
                                        ON  CREATE  SET  wl.product_quantity=$product_quantity,  wl.customer_email=$email
                                        """,
                                        wishlist_id=w[0],  product_quantity=w[1],  email=w[2]
                                )

wl_pop  =  PopulateWishlist(uri,  username,  password)
wl_pop.populate_wishlist()
wl_pop.close()
```

B.6 Populate Entity PaymentMethod

```
class  PopulatePaymentMethods:
        def  __init__(self,  uri,  user,  password):
                self.driver  =  GraphDatabase.driver(uri,  auth=(user,  password))
        def  close(self):
                self.driver.close()
        def  populate_payment_methods(self):
                payment_methods  =  [
                        (101,  1,  'john.doe@gmail.com'),
                        (102,  2,  'jane.smith@outlook.com'),
                        (103,  3,  'alice.jones@gmail.com'),
                        (104,  4,  'bob.brown@outlook.com'),
                        (105,  5,  'carol.white@outlook.com'),
                        (106,  6,  'laura.green@gmail.com'),
                        (107,  7,  'chris.lee@outlook.com'),
                        (108,  8,  'emily.davis@gmail.com'),
                        (109,  9,  'michael.jordan@yahoo.com'),
                        (110,  10,  'susan.martin@hotmail.com'),
                        (111,  11,  'nancy.parker@gmail.com'),
                        (112,  12,  'kevin.baker@gmail.com'),
```

```
                (113,  13,  'patricia.scott@yahoo.com'),
                (114,  14,  'frank.harris@gmail.com'),
                (115,  15,  'isabella.hernandez@gmail.com'),
                (116,  16,  'ryan.carter@gmail.com'),
                (117,  17,  'kimberly.allen@gmail.com'),
                (118,  18,  'hannah.collins@aol.com'),
                (119,  19,  'thomas.anderson@outlook.com'),
                (120,  20,  'elizabeth.james@gmail.com'),
                (121,  21,  'matthew.reed@hotmail.com'),
                (122,  22,  'mark.adams@gmail.com'),
                (123,  23,  'julia.moore@gmail.com'),
                (124,  24,  'sara.watson@outlook.com'),
                (125,  25,  'timothy.hall@gmail.com'),
                (126,  26,  'julia.moore@gmail.com'),
                (127,  27,  'joseph.cook@gmail.com'),
                (128,  28,  'aaron.morris@yahoo.com'),
                (129,  29,  'victoria.mitchell@gmail.com'),
                (130,  30,  'oliver.taylor@gmail.com'),
                (131,  31,  'joseph.cook@gmail.com'),
                (132,  32,  'michael.brown@gmail.com'),
                (133,  33,  'maria.hill@gmail.com'),
                (134,  34,  'diana.king@hotmail.com'),
                (135,  35,  'steven.wright@outlook.com'),
                (136,  36,  'henry.young@gmail.com'),
                (137,  37,  'megan.foster@outlook.com'),
                (138,  38,  'julia.moore@gmail.com'),
                (139,  39,  'timothy.hall@gmail.com'),
                (140,  40,  'julia.moore@gmail.com'),
                (141,  41,  'joseph.cook@gmail.com'),
                (142,  42,  'thomas.anderson@outlook.com'),
                (143,  43,  'laura.green@gmail.com'),
                (144,  44,  'kevin.baker@gmail.com'),
                (145,  45,  'sara.watson@outlook.com'),
                (146,  46,  'frank.harris@gmail.com')
        ]
        with  self.driver.session()  as  session:
                for  pm  in  payment_methods:
                        session.run(
                                """
                                MERGE  (p:PaymentMethod  {payment_method_id:$pmid})
                                ON  CREATE  SET  p.order_number=$order_number,  p.customer_email=$email
                                """,
                                pmid=pm[0],  order_number=pm[1],  email=pm[2]
                        )

pm_pop  =  PopulatePaymentMethods(uri,  username,  password)
pm_pop.populate_payment_methods()
pm_pop.close()
```

B.7 Populate Entity CreditCard

```
class  PopulateCreditCards:
        def  __init__(self,  uri,  user,  password):
                self.driver  =  GraphDatabase.driver(uri,  auth=(user,  password))
        def  close(self):
                self.driver.close()
        def  populate_credit_cards(self):
                credit_cards  =  [
                        (901,  '123',  '4112785638491723',  '2025-01',  'John  Doe',  'true',  101,  1),
                        (902,  '456',  '4228463190047812',  '2026-02',  'Jane  Smith',  'true',  102,  2),
                        (903,  '789',  '4337198052246111',  '2025-03',  'Alice  Jones',  'true',  103,  3),
                        (904,  '012',  '4442839056873291',  '2027-04',  'Bob  Brown',  'true',  104,  4),
                        (905,  '345',  '4559864120346753',  '2026-05',  'Carol  White',  'true',  105,  5),
                        (906,  '678',  '4663041752198742',  '2025-06',  'Laura  Green',  'true',  106,  6),
                        (907,  '901',  '4773918465129430',  '2024-07',  'Chris  Lee',  'true',  107,  7),
                        (908,  '234',  '4882956173402987',  '2025-08',  'Emily  Davis',  'true',  108,  8),
                        (909,  '567',  '4996127309456123',  '2026-09',  'Michael  Jordan',  'true',  109,  9),
                        (910,  '890',  '4007258391620748',  '2025-10',  'Susan  Martin',  'true',  110,  10),
                        (911,  '123',  '4112045678302916',  '2026-11',  'Nancy  Parker',  'true',  111,  11),
                        (912,  '456',  '4228391754062135',  '2025-12',  'Kevin  Baker',  'true',  112,  12),
                        (913,  '789',  '4337204890312347',  '2027-01',  'Patricia  Scott',  'true',  113,  13),
                        (914,  '012',  '4441293658712234',  '2026-02',  'Frank  Harris',  'true',  114,  14),
                        (915,  '345',  '4558967023148795',  '2025-03',  'Isabella  Hernandez',  'true',  115,  15),
                        (916,  '678',  '4662183947502349',  '2027-04',  'Ryan  Carter',  'true',  116,  16),
```

```
                (917, '901', '4779354812602348', '2026-05', 'Kimberly Allen', 'true', 117, 17),
                (918, '234', '4882975126301945', '2025-06', 'Hannah Collins', 'true', 118, 18),
                (919, '567', '4998134201579203', '2026-07', 'Thomas Anderson', 'true', 119, 19),
                (920, '890', '4002983746109471', '2025-08', 'Elizabeth James', 'true', 120, 20),
                (921, '123', '4117846192031742', '2026-09', 'Matthew Reed', 'true', 121, 21),
                (922, '456', '4229708156342098', '2025-10', 'Mark Adams', 'true', 122, 22),
                (923, '789', '4335129082734956', '2027-11', 'Julia Moore', 'true', 123, 23),
                (924, '012', '4441298456209183', '2026-12', 'Sara Watson', 'true', 124, 24),
                (925, '345', '4556748930175623', '2025-01', 'Timothy Hall', 'true', 125, 25),
                (926, '678', '4669135407620194', '2027-02', 'Julia Moore', 'true', 126, 26),
                (927, '901', '4771352046073895', '2026-03', 'Joseph Cook', 'true', 127, 27),
                (928, '234', '4882043795612870', '2025-04', 'Aaron Morris', 'true', 128, 28),
                (929, '567', '4997120348691753', '2026-05', 'Victoria Mitchell', 'true', 129, 29),
                (930, '890', '4002316789502347', '2025-06', 'Oliver Taylor', 'true', 130, 30),
                (931, '123', '4115689023745108', '2026-07', 'Joseph Cook', 'true', 131, 31),
                (932, '456', '4223047195062974', '2025-08', 'Michael Brown', 'true', 132, 32),
                (933, '789', '4338902473621409', '2027-09', 'Maria Hill', 'true', 133, 33),
                (934, '012', '4443072948712059', '2026-10', 'Diana King', 'true', 134, 34),
                (935, '345', '4559027310542893', '2025-11', 'Steven Wright', 'true', 135, 35)
        ]
        with self.driver.session() as session:
                for cc in credit_cards:
                        session.run(
                                """
                                MERGE (c:CreditCard {credit_card_id:$ccid})
                                ON CREATE SET c.verification_code=$verification_code, c.card_number=$card_number, c.cc_expiry_date=$cc_expiry
                                                c.name_on_card=$name_on_card, c.is_default=$is_default, c.payment_method_id=$payment
                                                c.order_number=$order_number
                                """,
                                ccid=cc[0], verification_code=cc[1], card_number=cc[2], cc_expiry_date=cc[3],
                                name_on_card=cc[4], is_default=cc[5], payment_method_id=cc[6], order_number=cc[7]
                        )

cc_pop = PopulateCreditCards(uri, username, password)
cc_pop.populate_credit_cards()
cc_pop.close()
```

B.8 Populate Entity GiftCard

```
class PopulateGiftCards:
        def __init__(self, uri, user, password):
                self.driver = GraphDatabase.driver(uri, auth=(user, password))
        def close(self):
                self.driver.close()
        def populate_gift_cards(self):
                gift_cards = [
                (801, 1500.00, 5000.00, '2025-12-31', 'GC20241001A', 136, 36),
                (802, 3000.00, 10000.00, '2026-06-30', 'GC20241002B', 137, 37),
                (803, 7000.00, 7500.00, '2026-11-15', 'GC20241003C', 138, 38),
                (804, 5000.00, 6000.00, '2025-07-25', 'GC20241004D', 139, 39),
                (805, 2000.00, 15000.00, '2026-08-20', 'GC20241005E', 140, 40),
                (806, 3000.00, 5000.00, '2025-09-15', 'GC20241006F', 141, 41),
                (807, 4500.00, 7000.00, '2027-01-05', 'GC20241007G', 142, 42),
                (808, 5000.00, 7500.00, '2026-04-10', 'GC20241008H', 143, 43),
                (809, 1200.50, 6000.00, '2025-03-30', 'GC20241009I', 144, 44),
                (810, 1200.00, 3000.00, '2026-10-15', 'GC20241010J', 145, 45),
                (811, 1900.00, 2500.00, '2025-05-05', 'GC20241011K', 146, 46)
                ]
                with self.driver.session() as session:
                        for gc in gift_cards:
                                session.run(
                                        """
                                        MERGE (g:GiftCard {gift_card_id:$gcid})
                                        ON CREATE SET g.current_balances=$current_balances, g.total_amount=$total_amount, g.gc_expiry_date=$gc_expiry
                                                        g.serial_number=$serial_number, g.payment_method_id=$payment_method_id, g.order_numb
                                        """,
                                        gcid=gc[0], current_balances=gc[1], total_amount=gc[2], gc_expiry_date=gc[3],
                                        serial_number=gc[4], payment_method_id=gc[5], order_number=gc[6]
                                )

gc_pop = PopulateGiftCards(uri, username, password)
gc_pop.populate_gift_cards()
gc_pop.close()
```

B.9 Populate Entity Return

```
class PopulateReturn:
    def __init__(self, uri, user, password):
        self.driver = GraphDatabase.driver(uri, auth=(user, password))
    def close(self):
        self.driver.close()
    def populate_return(self):
        returns = [
            (601, '2024-05-01', '2024-05-15', 'Completed', 19.13, 363.37, 4),
            (602, '2024-09-10', '2024-09-24', 'Pending', 16.28, 216.22, 19),
            (603, '2024-05-05', '2024-05-20', 'In Progress', 13.20, 250.80, 24),
            (604, '2024-10-01', '2024-10-15', 'Completed', 1.31, 42.34, 28),
            (605, '2024-11-01', '2024-11-15', 'Pending', 7.20, 352.80, 40)
        ]
        with self.driver.session() as session:
            for r in returns:
                session.run(
                    """
                    MERGE (ret:Return {ticket_number:$ticket_number})
                    ON CREATE SET ret.start_date=$start_date, ret.due_date=$due_date, ret.return_status=$return_status,
                                  ret.return_fee=$return_fee, ret.refund_total=$refund_total, ret.order_number=$order_
                    """,
                    ticket_number=r[0], start_date=r[1], due_date=r[2], return_status=r[3],
                    return_fee=r[4], refund_total=r[5], order_number=r[6]
                )

ret_pop = PopulateReturn(uri, username, password)
ret_pop.populate_return()
ret_pop.close()
```

B.10 Populate Entity Delivery

```
class PopulateDelivery:
    def __init__(self, uri, user, password):
        self.driver = GraphDatabase.driver(uri, auth=(user, password))
    def close(self):
        self.driver.close()
    def populate_delivery(self):
        deliveries = [
            (981234567, '2024-01-20', 'Delivered', 'john.doe@gmail.com', 1),
            (983456789, '2024-02-25', 'Delivered', 'jane.smith@outlook.com', 2),
            (987654321, '2024-03-15', 'Delivered', 'alice.jones@gmail.com', 3),
            (984321678, '2024-04-30', 'Pending', 'bob.brown@outlook.com', 4),
            (986789012, '2024-06-04', 'Delivered', 'carol.white@outlook.com', 5),
            (982345678, '2024-06-20', 'Pending', 'laura.green@gmail.com', 6),
            (988765432, '2024-07-28', 'Delivered', 'chris.lee@outlook.com', 7),
            (981234890, '2024-08-23', 'Delivered', 'emily.davis@gmail.com', 8),
            (982234567, '2024-09-10', 'Delivered', 'michael.jordan@yahoo.com', 9),
            (985678123, '2024-10-15', 'Pending', 'susan.martin@hotmail.com', 10),
            (987654310, '2024-02-01', 'Delivered', 'nancy.parker@gmail.com', 11),
            (981278456, '2024-03-20', 'Delivered', 'kevin.baker@gmail.com', 12),
            (982344567, '2024-03-31', 'Delivered', 'patricia.scott@yahoo.com', 13),
            (989876543, '2024-04-10', 'Pending', 'frank.harris@gmail.com', 14),
            (983456721, '2024-06-10', 'Delivered', 'isabella.hernandez@gmail.com', 15),
            (984567123, '2024-07-25', 'Delivered', 'ryan.carter@gmail.com', 16),
            (988765412, '2024-08-02', 'Delivered', 'kimberly.allen@gmail.com', 17),
            (981234678, '2024-08-20', 'Pending', 'hannah.collins@aol.com', 18),
            (983456098, '2024-09-15', 'Delivered', 'thomas.anderson@outlook.com', 19),
            (987123456, '2024-10-05', 'Pending', 'elizabeth.james@gmail.com', 20),
            (984210987, '2024-01-24', 'Delivered', 'matthew.reed@hotmail.com', 21),
            (983459876, '2024-02-17', 'Delivered', 'mark.adams@gmail.com', 22),
            (982345091, '2024-03-20', 'Delivered', 'julia.moore@gmail.com', 23),
            (981098765, '2024-05-01', 'Pending', 'sara.watson@outlook.com', 24),
            (987098123, '2024-06-01', 'Delivered', 'timothy.hall@gmail.com', 25),
            (983209874, '2024-06-10', 'Pending', 'julia.moore@gmail.com', 26),
            (981230765, '2024-07-21', 'Delivered', 'joseph.cook@gmail.com', 27),
            (982134590, '2024-08-25', 'Delivered', 'aaron.morris@yahoo.com', 28),
            (984312098, '2024-09-15', 'Delivered', 'victoria.mitchell@gmail.com', 29),
            (987654012, '2024-11-04', 'Pending', 'oliver.taylor@gmail.com', 30),
            (981034256, '2024-01-17', 'Delivered', 'joseph.cook@gmail.com', 31),
            (982345678, '2024-03-04', 'Delivered', 'michael.brown@gmail.com', 32),
            (984567890, '2024-03-08', 'Delivered', 'maria.hill@gmail.com', 33),
```

```
                (987651234, '2024-03-30', 'Delivered', 'diana.king@hotmail.com', 34),
                (981234099, '2024-05-15', 'Pending', 'steven.wright@outlook.com', 35),
                (983456072, '2024-06-29', 'Delivered', 'henry.young@gmail.com', 36),
                (984567210, '2024-07-29', 'Pending', 'megan.foster@outlook.com', 37),
                (982345167, '2024-08-21', 'Delivered', 'julia.moore@gmail.com', 38),
                (987098657, '2024-09-18', 'Delivered', 'timothy.hall@gmail.com', 39),
                (981276345, '2024-10-15', 'Pending', 'julia.moore@gmail.com', 40),
                (982314567, '2024-11-10', 'Delivered', 'joseph.cook@gmail.com', 41),
                (983457890, '2024-01-03', 'Delivered', 'thomas.anderson@outlook.com', 42),
                (984210987, '2024-02-10', 'Pending', 'laura.green@gmail.com', 43),
                (981034556, '2024-03-15', 'Delivered', 'kevin.baker@gmail.com', 44),
                (983209876, '2024-04-18', 'Delivered', 'sara.watson@outlook.com', 45),
                (982134578, '2024-06-01', 'Delivered', 'frank.harris@gmail.com', 46)
        ]
        with self.driver.session() as session:
            for d in deliveries:
                session.run(
                    """
                    MERGE (del:Delivery {track_number:$track_number})
                    ON CREATE SET del.date_of_delivery=$date_of_delivery, del.delivery_status=$delivery_status,
                                  del.customer_email=$email, del.order_number=$order_number
                    """,
                    track_number=d[0], date_of_delivery=d[1], delivery_status=d[2],
                    email=d[3], order_number=d[4]
                )

del_pop = PopulateDelivery(uri, username, password)
del_pop.populate_delivery()
del_pop.close()
```

B.11 Populate Entity Review

```
class PopulateReviews:
    def __init__(self, uri, user, password):
        self.driver = GraphDatabase.driver(uri, auth=(user, password))
    def close(self):
        self.driver.close()
    def populate_reviews(self):
        reviews = [
            (401, '2024-01-15', 'Great product, very satisfied!', 5, 'chris.lee@outlook.com', 50053),
            (402, '2024-01-17', 'Not worth the price, very disappointed.', 2, 'julia.moore@gmail.com', 50072),
            (403, '2024-01-19', 'Average quality, okay for the price.', 3, 'kevin.baker@gmail.com', 50011),
            (404, '2024-01-20', 'Fantastic! Exceeded my expectations.', 5, 'bob.brown@outlook.com', 50004),
            (405, '2024-01-22', 'Broke after a week, very poor quality.', 1, 'michael.jordan@yahoo.com', 50009),
            (406, '2024-01-24', 'I love it! Will buy again.', 4, 'steven.wright@outlook.com', 50035),
            (407, '2024-01-26', 'Not bad, does the job.', 3, 'maria.hill@gmail.com', 50033),
            (408, '2024-01-28', 'Very stylish and functional!', 5, 'emily.davis@gmail.com', 50008),
            (409, '2024-01-30', 'Terrible customer service experience.', 1, 'kimberly.allen@gmail.com', 50017),
            (410, '2024-02-01', 'Would not recommend, had issues.', 2, 'sara.watson@outlook.com', 50070),
            (411, '2024-02-03', 'Best purchase I've made this year!', 5, 'sara.watson@outlook.com', 50045),
            (412, '2024-02-05', 'Just okay, nothing special.', 3, 'sara.watson@outlook.com', 50091)
        ]
        with self.driver.session() as session:
            for rev in reviews:
                session.run(
                    """
                    MERGE (r:Review {review_number:$review_number})
                    ON CREATE SET r.review_date=$review_date, r.review_text=$review_text, r.ranking=$ranking,
                                  r.email=$email, r.product_number=$product_number
                    """,
                    review_number=rev[0], review_date=rev[1], review_text=rev[2], ranking=rev[3],
                    email=rev[4], product_number=rev[5]
                )

rev_pop = PopulateReviews(uri, username, password)
rev_pop.populate_reviews()
rev_pop.close()
```

B.12 Populate OrderedItem

```
class PopulateOrderedItems:
    def __init__(self, uri, user, password):
        self.driver = GraphDatabase.driver(uri, auth=(user, password))
```

```
def  close(self):
        self.driver.close()
def  populate_ordered_items(self):
        ordered_items  =  [
                (3001,  1,  50001,  1),
                (3002,  1,  50047,  1),
                (3003,  2,  50002,  1),
                (3004,  2,  50048,  1),
                (3005,  3,  50003,  1),
                (3006,  3,  50049,  1),
                (3007,  4,  50004,  1),
                (3008,  4,  50050,  1),
                (3009,  5,  50005,  2),
                (3010,  5,  50051,  1),
                (3011,  6,  50006,  1),
                (3012,  6,  50052,  3),
                (3013,  7,  50007,  4),
                (3014,  7,  50053,  1),
                (3015,  7,  50098,  1),
                (3016,  8,  50008,  1),
                (3017,  8,  50054,  1),
                (3018,  9,  50009,  1),
                (3019,  9,  50055,  1),
                (3020,  9,  50100,  1),
                (3021,  10,  50010,  1),
                (3022,  10,  50056,  1),
                (3023,  11,  50011,  1),
                (3024,  11,  50057,  1),
                (3025,  11,  50099,  2),
                (3026,  12,  50012,  1),
                (3027,  12,  50058,  1),
                (3028,  13,  50013,  1),
                (3029,  13,  50059,  1),
                (3030,  14,  50014,  1),
                (3031,  14,  50060,  1),
                (3032,  15,  50015,  1),
                (3033,  15,  50061,  1),
                (3034,  16,  50016,  2),
                (3035,  16,  50062,  1),
                (3036,  17,  50017,  1),
                (3037,  17,  50063,  1),
                (3038,  18,  50018,  1),
                (3039,  18,  50064,  1),
                (3040,  19,  50019,  1),
                (3041,  19,  50065,  1),
                (3042,  20,  50020,  4),
                (3043,  20,  50066,  1),
                (3044,  20,  50097,  1),
                (3045,  21,  50021,  1),
                (3046,  21,  50067,  1),
                (3047,  22,  50022,  1),
                (3048,  22,  50068,  1),
                (3049,  23,  50023,  1),
                (3050,  23,  50069,  1),
                (3051,  24,  50024,  1),
                (3052,  24,  50070,  1),
                (3053,  25,  50025,  1),
                (3054,  25,  50071,  2),
                (3055,  25,  50095,  1),
                (3056,  26,  50026,  1),
                (3057,  26,  50072,  1),
                (3058,  27,  50027,  1),
                (3059,  27,  50073,  1),
                (3060,  28,  50028,  1),
                (3061,  28,  50074,  1),
                (3062,  29,  50029,  1),
                (3063,  29,  50075,  1),
                (3064,  29,  50032,  1),
                (3065,  30,  50030,  1),
                (3066,  30,  50076,  1),
                (3067,  31,  50031,  1),
                (3068,  31,  50077,  1),
                (3069,  32,  50032,  1),
                (3070,  32,  50078,  1),
                (3071,  33,  50033,  1),
                (3072,  33,  50079,  1),
                (3073,  34,  50034,  1),
```

```
                (3074,  34,  50080,  1),
                (3075,  35,  50035,  1),
                (3076,  35,  50081,  1),
                (3077,  36,  50036,  1),
                (3078,  36,  50082,  1),
                (3079,  37,  50037,  3),
                (3080,  37,  50083,  1),
                (3081,  37,  50094,  2),
                (3082,  38,  50038,  1),
                (3083,  38,  50084,  1),
                (3084,  39,  50039,  1),
                (3085,  39,  50085,  1),
                (3086,  40,  50040,  1),
                (3087,  40,  50086,  1),
                (3088,  41,  50041,  1),
                (3089,  41,  50087,  1),
                (3090,  41,  50093,  1),
                (3091,  42,  50042,  1),
                (3092,  42,  50088,  1),
                (3093,  43,  50043,  1),
                (3094,  43,  50089,  1),
                (3095,  44,  50044,  1),
                (3096,  44,  50090,  1),
                (3097,  45,  50045,  1),
                (3098,  45,  50091,  1),
                (3099,  46,  50046,  1),
                (3100,  46,  50092,  1)
        ]
        with  self.driver.session()  as  session:
                for  oi  in  ordered_items:
                        session.run(
                                """
                                MERGE  (oitem:OrderedItem  {ordered_item_id:$oid})
                                ON  CREATE  SET  oitem.order_number=$order_number,  oitem.product_number=$product_number,
                                                        oitem.ordered_quantity=$ordered_quantity
                                """,
                                oid=oi[0],  order_number=oi[1],  product_number=oi[2],  ordered_quantity=oi[3]
                        )

oi_pop  =  PopulateOrderedItems(uri,  username,  password)
oi_pop.populate_ordered_items()
oi_pop.close()
```

B.13 Create Relationships

```
class  PopulateRelationships:
        def  __init__(self,  uri,  user,  password):
                self.driver  =  GraphDatabase.driver(uri,  auth=(user,  password))
        def  close(self):
                self.driver.close()

        def  create_relationships(self):
                with  self.driver.session()  as  session:
                        #  Customer-Order
                        session.run("""
                        MATCH  (c:Customer),  (o:Order)
                        WHERE  o.customer_email  =  c.email
                        MERGE  (c)-[:PLACE]->(o)
                        """)

                        #  Customer-Wishlist
                        session.run("""
                        MATCH  (c:Customer),  (w:Wishlist)
                        WHERE  w.customer_email  =  c.email
                        MERGE  (c)-[:REGISTERS_FOR]->(w)
                        """)

                        #  PaymentMethod-Order  &  PaymentMethod-Customer
                        session.run("""
                        MATCH  (c:Customer),  (o:Order),  (pm:PaymentMethod)
                        WHERE  pm.order_number  =  o.order_number  AND  pm.customer_email  =  c.email
                        MERGE  (o)-[:IS_PAID_BY]->(pm)
                        MERGE  (c)-[:USES]->(pm)
                        """)
```

```
                        #  CreditCard-PaymentMethod
                        session.run("""
                        MATCH  (pm:PaymentMethod),  (cc:CreditCard)
                        WHERE  cc.payment_method_id  =  pm.payment_method_id
                        MERGE  (pm)-[:USES]->(cc)
                        """)

                        #  GiftCard-PaymentMethod
                        session.run("""
                        MATCH  (pm:PaymentMethod),  (gc:GiftCard)
                        WHERE  gc.payment_method_id  =  pm.payment_method_id
                        MERGE  (pm)-[:USES]->(gc)
                        """)

                        #  Order-OrderedItem
                        session.run("""
                        MATCH  (o:Order),  (oi:OrderedItem)
                        WHERE  oi.order_number  =  o.order_number
                        MERGE  (o)-[:CONTAINS]->(oi)
                        """)

                        #  OrderedItem-Product
                        session.run("""
                        MATCH  (oi:OrderedItem),  (p:Product)
                        WHERE  oi.product_number  =  p.product_number
                        MERGE  (oi)-[:IS]->(p)
                        """)

                        #  Product-Category
                        session.run("""
                        MATCH  (p:Product),  (cat:Category)
                        WHERE  p.category_id  =  cat.category_id
                        MERGE  (p)-[:BELONGS_TO]->(cat)
                        """)

                        #  Delivery-Customer  and  Delivery-Order
                        session.run("""
                        MATCH  (d:Delivery),  (c:Customer),  (o:Order)
                        WHERE  d.customer_email  =  c.email  AND  d.order_number  =  o.order_number
                        MERGE  (d)-[:DELIVERS_TO]->(c)
                        MERGE  (d)-[:IS_GENERATED_BY]->(o)
                        """)

                        #  Return-Order
                        session.run("""
                        MATCH  (r:Return),  (o:Order)
                        WHERE  r.order_number  =  o.order_number
                        MERGE  (r)-[:IS_RETURNED]->(o)
                        """)

                        #  If  product  has  ticket_number  (return),  link  product  to  return
                        session.run("""
                        MATCH  (p:Product),  (r:Return)
                        WHERE  p.ticket_number  =  r.ticket_number
                        MERGE  (r)-[:IS_RETURNED_FOR]->(p)
                        """)

                        #  Product-Wishlist  if  wishlist_id  not  null
                        session.run("""
                        MATCH  (p:Product),  (w:Wishlist)
                        WHERE  p.wishlist_id  =  w.wishlist_id
                        MERGE  (w)-[:CONTAINS]->(p)
                        """)

                        #  Review-Customer  and  Review-Product
                        session.run("""
                        MATCH  (rev:Review),  (c:Customer),  (p:Product)
                        WHERE  rev.email  =  c.email  AND  rev.product_number  =  p.product_number
                        MERGE  (c)-[:WRITES]->(rev)
                        MERGE  (rev)-[:IS_WRITTEN_FOR]->(p)
                        """)

rel_pop  =  PopulateRelationships(uri,  username,  password)
rel_pop.create_relationships()
rel_pop.close()
```

B.14 The screenshot of my neo4j database:

I use this Cypher query to display all outputs
MATCH (n)-[r]->(m)
RETURN n, r, m

(sometimes the image is not properly displayed on Github, you may want to download the repository as .zip and open this notebook in vscode. Otherwise, check the file named "48372_Q2B_ScreenShot_of_neo4j")

Question (C)/(2C)

I need to produce a list of all customers and their orders, including the list of products in each order and the (grand) total paid. Followed by showing the customer's name,email, order number, and order date.

First, I will look into which tables contains the information I need, and for this query, they are **Customer, Order, OrderedItem, and Product**. Thus, I will use Cypher language **MATCH** to "link" them together.

Then I just need to get all information I need by using **RETURN**. For most information I can directly refer to properties inside nodes. However, since I need to produce a list of products purchased, and the list does not initially exist, I will aggregate all **short_name** properties by **COLLECT()** to produce a list.

Finally, group them by the alphabet sequence From A to Z.

By comparing the answer I get here to the answer I get from first assignment using SQL, I can confirm that this is the right approach.

```
driver = GraphDatabase.driver(uri, auth=(username, password))

def list_customers_orders_products(driver):
        query = """
        MATCH (c:Customer)-[:PLACE]->(o:Order)-[:CONTAINS]->(oi:OrderedItem)-[:IS]->(p:Product)
        WITH c, o, COLLECT(p.short_name) AS products
        RETURN c.name AS CustomerName,
                       c.email AS CustomerEmail,
                       o.order_number AS OrderNumber,
                       o.order_date AS OrderDate,
                       products AS Products,
                       o.grand_total AS GrandTotal
        ORDER BY CustomerName
        """

        with driver.session() as session:
                results = session.run(query)
                for record in results:
                        print("Customer Name:", record["CustomerName"])
                        print("Customer Email:", record["CustomerEmail"])
                        print("Order Number:", record["OrderNumber"])
                        print("Order Date:", record["OrderDate"])
                        print("Products:", record["Products"])
                        print("Grand Total:", record["GrandTotal"])
# use - to bloack different sections.
                        print("-" * 40)

list_customers_orders_products(driver)
driver.close()
```

```
Customer Name: Aaron Morris
Customer Email: aaron.morris@yahoo.com
Order Number: 28
Order Date: 2024-08-22
Products: ['Tennis Racket', 'Camping Stove']
Grand Total: 155.18
----------------------------------------
Customer Name: Alice Jones
Customer Email: alice.jones@gmail.com
Order Number: 3
Order Date: 2024-03-10
Products: ['Headphones', 'Desk']
Grand Total: 499.98
----------------------------------------
Customer Name: Bob Brown
```

```
Customer Email: bob.brown@outlook.com
Order Number: 4
Order Date: 2024-04-25
Products: ['Microwave Oven', 'Tablet']
Grand Total: 407.98
----------------------------------------
Customer Name: Carol White
Customer Email: carol.white@outlook.com
Order Number: 5
Order Date: 2024-05-30
Products: ['Blender', 'Garden Chair']
Grand Total: 228.98
----------------------------------------
Customer Name: Chris Lee
Customer Email: chris.lee@outlook.com
Order Number: 7
Order Date: 2024-07-22
Products: ['Sofa', 'Baking Set', 'Playmat']
Grand Total: 1982.593
----------------------------------------
Customer Name: Diana King
Customer Email: diana.king@hotmail.com
Order Number: 34
Order Date: 2024-03-28
Products: ['Leather Jacket', 'Portable Charger']
Grand Total: 229.98
----------------------------------------
Customer Name: Elizabeth James
Customer Email: elizabeth.james@gmail.com
Order Number: 20
Order Date: 2024-10-02
Products: ['Camping Lantern', 'Smart Home Hub', 'Baby Clothes']
Grand Total: 242.95
----------------------------------------
Customer Name: Emily Davis
Customer Email: emily.davis@gmail.com
Order Number: 8
Order Date: 2024-08-18
Products: ['Coffee Table', 'Electric Toothbrush']
Grand Total: 269.98
----------------------------------------
Customer Name: Frank Harris
Customer Email: frank.harris@gmail.com
```

Question (D)/(2D)

In this question, I am required to list the e-mail, name, birthday, gift card balance, and list of products in baskets from customers who have items in their baskets. To fulfill this demand, I need the information from table Customer, Wishlist and Product. Like question C, I will link them together.

Also following the same logic in the question c, I create a list of products. But this time, I only need to create lists for products inside wishlists. Thus, I will use **p.wishlist_id = w.wishlist_id** to only include those products.

I also need to find out the gift card balances (if there is any). Since **some customers have wishlists but do not possess gift cards**, I will default NULL output into 0 by **SUM(gc.current_balances),0)**. Of course, if a customer has gift card, the value will be printed.

By comparing the answer I get here to the answer I get from first assignment using SQL, I can confirm that this is the right approach.

```
driver = GraphDatabase.driver(uri, auth=(username, password))

def list_customers_with_baskets_and_giftcards(driver):
    query = """
    MATCH (c:Customer)-[:REGISTERS_FOR]->(w:Wishlist)
    MATCH (p:Product)
    WHERE p.wishlist_id = w.wishlist_id
    WITH c, w, COLLECT(p.short_name) AS products
    OPTIONAL MATCH (c)-[:USES]->(:PaymentMethod)-[:USES]->(gc:GiftCard)
    WITH c, products, COALESCE(SUM(gc.current_balances),0) AS GiftCardBalance
    RETURN c.email AS Email,
                c.name AS CustomerName,
                c.birthdate AS Birthdate,
                GiftCardBalance,
                products AS Products
    ORDER BY CustomerName
    """

    with driver.session() as session:
        results = session.run(query)
```

```
            for record in results:
                print("Email:", record["Email"])
                print("Name:", record["Name"])
                print("Birthdate:", record["Birthdate"])
                print("GiftCardBalance:", record["GiftCardBalance"])
                print("Products in Basket:", record["Products"])
# use - to bloack different sections.
                print("-" * 40)

list_customers_with_baskets_and_giftcards(driver)
driver.close()
```

```
Received notification from DBMS server: {severity: WARNING} {code: Neo.ClientNotification.Statement.AggregationSkippedNull} {category: UNRECOGNIZE
Email: carol.white@outlook.com
Name: Carol White
Birthdate: 1995-11-20
GiftCardBalance: 0
Products in Basket: ['Smartphone Case']
----------------------------------------
Email: elizabeth.james@gmail.com
Name: Elizabeth James
Birthdate: 1991-10-30
GiftCardBalance: 0
Products in Basket: ['Smartphone', 'Cordless Drill']
----------------------------------------
Email: emily.davis@gmail.com
Name: Emily Davis
Birthdate: 1992-12-08
GiftCardBalance: 0
Products in Basket: ['Headphones', 'Fiction Novel', 'Air Conditioner']
----------------------------------------
Email: frank.harris@gmail.com
Name: Frank Harris
Birthdate: 1980-04-14
GiftCardBalance: 1900.0
Products in Basket: ['Sunscreen', 'Vitamins']
----------------------------------------
Email: hannah.collins@aol.com
Name: Hannah Collins
Birthdate: 1989-12-20
GiftCardBalance: 0
Products in Basket: ['Smart Home Hub']
----------------------------------------
Email: isabella.hernandez@gmail.com
Name: Isabella Hernandez
Birthdate: 1995-02-06
GiftCardBalance: 0
Products in Basket: ['Face Cream', 'Air Fryer']
----------------------------------------
Email: isabella.hernandez@gmail.com
Name: Isabella Hernandez
Birthdate: 1995-02-06
GiftCardBalance: 0
Products in Basket: ['Inflatable Kayak']
----------------------------------------
Email: julia.moore@gmail.com
Name: Julia Moore
Birthdate: 1991-01-11
GiftCardBalance: 9000.0
Products in Basket: ['Laptop', 'Sofa', 'Watch', 'Fitness Tracker', 'Board Game', 'Action Camera', 'Coffee Maker', 'Frying Pan', 'Tent', 'Smart Wat
----------------------------------------
Email: julia.moore@gmail.com
Name: Julia Moore
Birthdate: 1991-01-11
GiftCardBalance: 9000.0
Products in Basket: ['Soccer Ball', 'Kids Bed']
----------------------------------------
Email: julia.moore@gmail.com
Name: Julia Moore
```

Question (E)/(2E)

For this question, I will find the top 2 products that have most sales in their category. I will print their product names, corresponding category name and the exact number for their sales.

By observing data structure, I know that these tables are needed: Category, Product, OrderedItem, and Order. I link them together.

Then, I calculate the sales quantity of each product by summarizing the ordered_quantity of products from table OrderedItem. I set this as totalsold. After that, I will order them(products) with descending order from high sales quantity to low sales quantity.

However, this is not finished. In the printout I need product names, category names, and sales. I can leave the category names aside, but I need to combine the product name and sales together because they are in a 1-to-1 relation. Every product needs their own sales. Thus, still using the same approach from previous questions, I combine them into a list: **COLLECT({productName: p.short_name, totalsold: totalsold})**, and I name it as ProductSales. Because I only need the first 2, so I **slice** the list ProductSales, and only leave the first 2 items: **productSales[0..2]**. I name this as TopTwoProducts.

Finally, for interpretation, just to make it more visually friendly, I will print the category name, and then the 2 top sales products inside that category. This is done by a simple for loop. Apart from that, just like what I did in previous questions, I use "-" to split different sections.

By comparing the answer I get here to the answer I get from first assignment using SQL, I can confirm that this is the right approach.

```
driver = GraphDatabase.driver(uri, auth=(username, password))

def top_two_products_per_category(driver):
    # Define the Cypher query to fetch the top two products per category
    query = """
    MATCH (cat:Category)<-[:BELONGS_TO]-(p:Product)<-[:IS]-(oi:OrderedItem)<-[:CONTAINS]-(o:Order)
    WITH cat, p, SUM(oi.ordered_quantity) AS totalsold
    ORDER BY cat.category_type, totalsold DESC
    WITH cat, COLLECT({productName: p.short_name, totalsold: totalsold}) AS productSales
    RETURN cat.category_type AS Category, productSales[0..2] AS TopTwoProducts
    ORDER BY Category
    """

    # Execute the query and print results
    with driver.session() as session:
        results = session.run(query)
        for record in results:
            print("Category:", record["Category"])
            for product in record["TopTwoProducts"]:
                print(f"   Product: {product['productName']}, Units Sold: {product['totalsold']}")
# use - to bloack different sections/categories.
            print("-" * 40)

top_two_products_per_category(driver)
driver.close()
```

```
Category: Automotive
  Product: Sleeping Bag, Units Sold: 2
  Product: Car Seat Cover, Units Sold: 1
----------------------------------------
Category: Books
  Product: Fiction Novel, Units Sold: 1
  Product: Cookbook, Units Sold: 1
----------------------------------------
Category: Electronics
  Product: Smartphone, Units Sold: 1
  Product: Laptop, Units Sold: 1
----------------------------------------
Category: Fashion
  Product: Refrigerator, Units Sold: 1
  Product: T-Shirt, Units Sold: 1
----------------------------------------
Category: Furniture
  Product: Sofa, Units Sold: 4
  Product: Kids Bed, Units Sold: 2
----------------------------------------
Category: Health & Beauty
  Product: Sunscreen, Units Sold: 1
  Product: Hair Dryer, Units Sold: 1
----------------------------------------
Category: Home Appliances
  Product: Smart Bulb, Units Sold: 3
  Product: Blender, Units Sold: 2
----------------------------------------
Category: Jewelry & Accessories
  Product: Watch, Units Sold: 1
  Product: Smart Watch, Units Sold: 1
----------------------------------------
Category: Music & Entertainment
  Product: Router, Units Sold: 1
  Product: Video Game, Units Sold: 1
----------------------------------------
Category: Office Supplies
  Product: Notebook, Units Sold: 1
  Product: Printer, Units Sold: 1
----------------------------------------
Category: Sports & Outdoors
  Product: Camping Lantern, Units Sold: 4
```

```
  Product: Soccer Ball, Units Sold: 2
----------------------------------------
Category: Toys & Games
  Product: Puzzle, Units Sold: 3
  Product: Board Game, Units Sold: 1
----------------------------------------
```

This code cell is used for **deleting the database**. This is because sometimes I want to test my code, and see what is the interpretation of my code in the neo4j. But I need to empty the database before every test or otherwise there will be duplicate data. The following code can delete almost all data, which enables me checking my code.

```
class Neo4jGraphModel:
    def __init__(self, uri, user, password):
        self.driver = GraphDatabase.driver(uri, auth=(user, password))

    def close(self):
        self.driver.close()

    def clear_all_data(self):
        with self.driver.session() as session:
            session.run("MATCH (n) DETACH DELETE n")


graph = Neo4jGraphModel(uri, username, password)
graph.clear_all_data()
graph.close()
```